

1 Introduction

During this sprint, we worked a lot on the project. After the last sprint, we decided to meet in the lab every morning and work together. This decision helped us become a stronger team and achieve better results. We successfully added navigation through the track. We also improved line detection, line following, and object detection. We added new images and classes to the dataset, made the system detect and respond to traffic signs, and found and fixed a bug in the platform's code that was slowing down the system. At the end of the sprint, we found a problem with pedestrian detection. The system only uses the upper right side of the frame for sign detection, so it cannot see pedestrians on the left side of the road. We started fixing this issue and will complete it in the next sprint.

2 Planned activities

During this sprint, we had a plan to accomplish the following tasks:

- Fix latency issues
- Collect images from the car camera
- Traffic sign detection
- Intersection traversal
- Begin working on path planning
- Possible minor hardware improvements

3 Status of planned activities

- **Fix latency issues**
The gateway initially caused significant latency. Small optimizations helped briefly but failed under heavy load, prompting a refactor to improve throughput and address persistent issues. A key focus was optimizing the lastOnly subscriber method, reducing redundant processing. The current implementation performs well and we have backup strategies available if the system fails under heavier loads. Network congestion in the Flask server led to an attempt at a custom server, but results were unsatisfactory. Instead, the lessons we learned from that process were used to optimize the Flask server, achieving efficient video streaming with minimal latency.
- **Collect images from the car camera**
We let the car drive around on its own and also drove it around manually to collect images from as many angles and with as many lighting changes as possible. As of right now, we have little more than a thousand images in our dataset.
- **Traffic sign detection**
After collecting all of our images, we annotated them and trained our YOLO11 model. Having deployed it on the platform and testing it, we have no complaints for right now. The car can successfully detect and react to signs. However, while we have no issue with handling traffic signs, we still need to fix our pedestrian detection issue. The main idea is to have two separate YOLO11 models working in parallel. We are looking at different solutions as well, but that is currently how we intend to fix the issue.
- **Begin working on path planning**
To our surprise, we completed the path planning module very quickly, only needing a few days to do so. The provided .graphml file was of great help and with some minor tweaks everything works now. We also decided to map out the test track and write our own .graphml file for it so that we could test the path planning on what we have.

- **Intersection traversal**

Combining our path planning and sign detection module, the car is able to handle intersections. On occasion we have issues with turning due to the car coming to the intersection at an awkward angle. This is mostly due to the small test track having some weird turns, but we've begun working on a fix just in case something like this happens on the competition track too.

- **Possible minor hardware improvements**

To enhance the system's perception and decision-making, several hardware upgrades were made. We added three ultrasonic sensors; one at the front for obstacle detection and front car speed matching, and two at the sides for detecting free parking spaces and potentially assisting with intersection merging. Along with that, we added an ESP32 to manage these sensors, filter their data and transmit that information to the RaspberryPi.

4 General status of the project

The car can drive itself on the track, it follows its lane, detects and reacts to signs, handles intersections and follows a given path. It also slows down when taking turns and intersections, and moves slowly when going over a pedestrian crossing. We've trained our YOLO11 model, it detects signs, traffic lights and pedestrians well. We still haven't gotten around to reacting to traffic lights and pedestrians yet because those are the last things we added to our dataset. We also realized that some roads might be closed, so we intend to develop a way to replan our path on the go. Apart from that, we've also written a script for the car to follow when parking where we use a predefined routine. The parking has been tested on the track and works well. The added ESP32 and sensors are already integrated into the decision making and should tell the car not to try and park on a space that's already taken. That being said, we have yet to attach the sensors to the platform, so we haven't tested that functionality yet.

5 Upcoming activities

In the next sprint we want to do the following tasks:

- Optimize where possible
- Traffic light and pedestrian reaction
- Handle overtaking
- Place additional sensors on platform
- Handle obstacle avoidance
- Path replanning